Università degli Studi di Salerno

Corso di Ingegneria del Software

UNISA-TCG

Object Design Document

Data: 17/02/2026

Coordinatore del progetto:

Nome	Matricola
Sepe Giuseppe	0512119386

Partecipanti:

Nome	Matricola
Senatore Francesco	0512120568
Sepe Giuseppe	0512119386
Sullo Diego	0512119455

Scritto da:	Senatore Francesco, Sepe Giuseppe, Sullo Diego

Revision History

Data	Versione	Descrizione	Autore
22/12/2025	**1.0**	**Object Design Document**	**Francesco Senatore, Diego Sullo, Giuseppe Sepe**
28/12/2025	**1.1**	**Apportate modifiche legate alla Category Partition**	**Francesco Senatore, Diego Sullo, Giuseppe Sepe**
17/02/2026	**1.2**	**Modifiche finali**	**Francesco Senatore, Diego Sullo, Giuseppe Sepe**

Indice

1. Introduzione..... 5
1.1.Object Design Tradeoffs..... 5
1.1.1 Durability vs platform dependance.....5
1.1.2 Performance versus flexibility..... 5
1.2 Interface documentation guidelines..... 5
1.3 Definitions, acronyms, and abbreviations..... 6
1.4 References.....8
2. Packages..... 9
2.1 it.unisatcg.model..... 9
2.2 it.unisatcg.control.....9
2.3 Web Content (Struttura del Presentation Layer).....9
2.4 Dipendenza tra Package.....9
3. Class interfaces Glossary.....10
3.1 Package it.unisatcg.model (Bean)..... 10
Classe Prodotto.....10
Classe Categoria.....10
Classe Utente.....11
Classe Ordine.....11
Classe DettaglioOrdine..... 12
Classe Recensione..... 12
Classe ArticoloCarrello.....12
Classe Carrello.....13
Classe Utility: DBConnection..... 14
3.2 Package it.unisatcg.model.dao (DAO).....15
Interfaccia ProdottoDAO..... 15
Interfaccia CategoriaDAO..... 15
Interfaccia UtenteDAO..... 16
Interfaccia OrdineDAO.....17
Interfaccia DettaglioOrdineDAO.....18
Interfaccia RecensioneDAO.....18
3.3 Package it.unisatcg.control (Servlet)..... 19
CategoriaProdottiServlet.....19
ProductDetailServlet..... 19
ImageServlet.....19
LogoutServlet.....19

RegisterServlet....20
UpdateProfileServlet....20
UserOrdersServlet....20
AdminOrderDetailServlet....20
RecensioneServlet....21
RicercaServlet....21
NewProductServlet....21
CheckoutServlet....21
GestioneCarrelloServlet....22
AdminProductsServlet....22
LoginServlet....22
AdminOrdiniServlet....23
AdminRecensioniServlet....23
EditProductServlet....23
3.4 Componenti di Sicurezza (Filtri)....24
AdminFilter....24
UserFilter....24

1. Introduzione

Il presente **Object Design Document (ODD)** definisce la progettazione a livello di oggetto della piattaforma e-commerce **UNISA-TCG**. Questo documento funge da ponte tra l'architettura di sistema descritta nel System Design Document (SDD) e l'effettiva implementazione del software. La progettazione si basa rigorosamente sul pattern architetturale **Model-View-Controller (MVC)**, garantendo una netta separazione tra la rappresentazione dei dati, la logica di business e l'interfaccia utente.

Viene descritta la struttura interna dei componenti del sistema, focalizzandosi su come gli oggetti collaborano per fornire le funzionalità di e-commerce (es. gestione del catalogo, carrello e checkout sicuro).soddisfare i requisiti funzionali identificati nel RAD.

1.1.Object Design Tradeoffs

Il sistema utilizza lo standard **JDBC** per l'interfacciamento con il DBMS **MySQL**.

- **Vantaggio:** Garantisce un'elevata durabilità e integrità dei dati tramite proprietà **ACID**.
- **Trade-off:** L'uso di query SQL specifiche per MySQL e la configurazione dell'hosting locale (On-Premise) legano parzialmente il software alla piattaforma scelta, ma assicurano il pieno controllo dell'infrastruttura necessario per i backup e il tuning delle performance.

1.1.1 Durability vs platform dependance

La scelta di **JDBC** e **MySQL** garantisce un'elevata integrità dei dati (proprietà ACID) al costo di un legame specifico con un sistema di gestione di database relazionali.

1.1.2 Performance versus flexibility

Per rispettare il requisito di tempi di risposta inferiori a un secondo (DG_2), si è scelto di:

- Limitare la complessità dei filtri di ricerca iniziali (Usabilità e Flessibilità).
- Accettare una potenziale **denormalizzazione** dei dati nel database per velocizzare le query di ricerca sul catalogo, riducendo la flessibilità di schemi puramente relazionali in favore della velocità di esecuzione.

1.2 Interface documentation guidelines

Oltre alle convenzioni già citate (nomi singolari per le classi, verbi per i metodi), aggiungiamo le seguenti specifiche per il progetto UNISA-TCG:

- **Pattern DAO:** Ogni accesso al database deve avvenire esclusivamente tramite interfacce DAO (es. ProdottoDAO), restituendo oggetti POJO (Plain Old Java Objects).
- **Separazione L1/L2:** Gli oggetti di interfaccia (JSP) non devono contenere logica di business, che è delegata interamente alle Servlet e ai Controller (es. Gestione Carrello Control).

1.3 Definitions, acronyms, and abbreviations

Termine/Acronimo	Definizione
ACID	(Atomicity, Consistency, Isolation, Durability). Proprietà fondamentali delle transazioni di database che garantiscono l'integrità dei dati, cruciali per il **Checkout** (UC3).
CRUD	(Create, Read, Update, Delete). Le quattro operazioni fondamentali di manipolazione dei dati utilizzate dal sottosistema **Persistenza Dati** e gestite dal **Gestore** sul Catalogo (UC5).
DAO	**Data Access Object**. Pattern software (Layer di Astrazione) che gestisce l'accesso ai dati, separando la logica di business dal database (L3).
DBMS	**Database Management System**. Software utilizzato per gestire e amministrare il database relazionale (**MySQL**).
GDPR	**General Data Protection Regulation** (Regolamento UE 2016/679). Normativa sulla protezione dei dati personali a cui il sistema deve conformarsi.
HTTPS	**Hypertext Transfer Protocol Secure**. Protocollo di comunicazione utilizzato per crittografare i dati in transito tra L1 e L2.
JDBC	**Java Database Connectivity**. API standard Java utilizzata dall'Application Server (L2) per connettersi al Database Server (L3).

L1, L2, L3	I tre livelli dell'architettura (3-Tier): **L1** (Presentazione), **L2** (Logica Applicativa), **L3** (Dati).
PSP	**Payment Service Provider**. Fornitore di servizi di pagamento esterno (simulato) utilizzato per autorizzare le transazioni finanziarie.
RAD	**Requirements Analysis Document**. Documento che precede l'SDD e contiene la specifica dei requisiti e i modelli (Classi, Casi d'Uso, Sequenza).
Rollback	Operazione eseguita dal DBMS che annulla tutte le modifiche apportate durante una transazione in caso di fallimento (Gestione Errori 3.7).
SDD	**System Design Document**. Il documento che specifica l'architettura e la progettazione del sistema.
TCG	**Trading Card Game**. Il tipo di prodotto gestito dal sistema (es. carte collezionabili).
Tomcat	**Apache Tomcat**. L'Application Server (container di servlet) utilizzato come ambiente di esecuzione per la logica L2.
UC	**Use Case** (Caso d'Uso). Sequenza di azioni definita nel RAD (es. UC3: Checkout e Pagamento).

Uptime	Percentuale di tempo in cui il sistema è completamente operativo e disponibile agli utenti.
WAR	**Web Archive**. Formato di file utilizzato per impacchettare i componenti web dell'applicazione per il deployment su Tomcat.
BLOB	**Binary Large Object.** Tipo di dato usato nei database per memorizzare grandi quantità di dati binari non strutturati, spesso sono immagini, video, audio o file compressi.

1.4 References

- Requirements Analysis Document (RAD) UNISA-TCG.
- System Design Document (SDD) UNISA-TCG.

2. Packages

2.1 it.unisatcg.model

Questo package costituisce il cuore dei dati del sistema. È l'unico che interagisce direttamente con il database.

- **Bean (Entity):** Classi come **Prodotto.java**, **Utente.java**, **Categoria.java**, **Ordine.java**. Esse rappresentano le tabelle del database sotto forma di oggetti Java (POJO).
- **DAO (Data Access Object):** Interfacce e implementazioni (es. P**rodottoDAO.java**, **ProdottoDAOImp.java**). Contengono la logica SQL (all interno di un sotto-package "dao").
- **Gestione Connessione:** La classe **DBConnection.java** risiede qui ed è utilizzata dai DAO per ottenere il riferimento al database MySQL.

2.2 it.unisatcg.control

Questo package implementa il ruolo di Controller nel pattern MVC. Contiene tutte le Servlet che mediano tra l'utente e il modello.

- Servlet principali: **LoginServlet**, **CategoriaProdottiServlet**, **CheckoutServlet**, **RegistrationServlet**, ecc.
- Ruolo: Estraggono i parametri dalle richieste HTTP, invocano i metodi dei DAO nel package model e decidono a quale pagina JSP fare il forward dei risultati.

2.3 Web Content (Struttura del Presentation Layer)

Rappresenta il **Livello 1 (Vista)**. I file sono organizzati nella root delle risorse web:

- **Pagine JSP:** index.jsp, products.jsp, cart.jsp, ecc. (Generazione dinamica dell'HTML).
- **Asset Statici:** style.css (Style), main.js, ed eventuali ulteriori file javascript o css (Logica client-side).
- **Ruolo:** Visualizzare i dati all'utente e inviare input alle Servlet nel package control.

2.4 Dipendenza tra Package

La gerarchia delle dipendenze segue un flusso unidirezionale **top-down** per garantire il disaccoppiamento tra i livelli e prevenire riferimenti circolari:

- **it.unisatcg.control** dipende da **it.unisatcg.model**: le Servlet (Controller) utilizzano le interfacce DAO per l'accesso ai dati e le classi Bean per incapsulare le informazioni da mostrare nelle viste (JSP).
- **it.unisatcg.model (DAO)** dipende dalle classi **Bean** e da **DBConnection**: le implementazioni DAO (DAOImp) utilizzano i Bean per mappare i record del database in oggetti Java e si appoggiano alla classe DBConnection per la gestione della connettività JDBC e del pool di connessioni.
- **it.unisatcg.model (Bean)**: le classi di dominio (es. Prodotto, Utente) sono **indipendenti**. Non hanno dipendenze verso altri pacchetti del sistema, garantendo che la struttura dei dati sia isolata dalla logica di persistenza e di controllo.

3. Class interfaces Glossary

In questa sezione vengono dettagliate le interfacce pubbliche delle classi appartenenti allo stralcio del sistema relativo alla **Visualizzazione del catalogo e dei prodotti singoli**. Per ogni classe vengono indicati gli attributi principali, le operazioni disponibili e le eccezioni sollevate.

3.1 Package it.unisatcg.model (Bean)

Classe Prodotto

Rappresenta l'entità fondamentale del catalogo (carta collezionabile o accessorio).

- **Attributi:**
  - **private int id**: Identificatore univoco del prodotto.
  - **private String nome**: Nome del prodotto.
  - **private String descrizione**: Descrizione dell'articolo.
  - **private double prezzo**: Prezzo di vendita.
  - **private int quantita**: Numero di pezzi disponibili in magazzino.
  - **private int categoriaId**: Riferimento alla categoria di appartenenza.
  - **private boolean isDisponibile**: Definisce se un prodotto è disponibile o esaurito
  - **private String specifiche**: Descrizione più dettagliata del prodotto
  - **private byte[] foto**: BLOB contenente l'immagine del prodotto.
  - **private int venditoreId**: Identificatore del venditore per distinguere prodotti del negozio da prodotti aggiunti da utenti
- **Metodi:**
  - **Getter/Setter**: Metodi standard per l'accesso e la modifica degli attributi sopra elencati.

Classe Categoria

Definisce la tipologia del prodotto (Es. Pokemon, Accessori, Magic).

- **Attributi:**
  - **private int id**: Identificatore della categoria.
  - **private String nome**: Nome della categoria.
- **Metodi:**
  - **Getter/Setter**: Metodi standard per la gestione dell'entità.

Classe Utente

Rappresenta l'utente della piattaforma, distinguendo tra acquirenti semplici e amministratori del sistema (Gestori).

- **Attributi:**
  - **private int id:** Identificatore univoco dell'utente.
  - **private String nome:** Nome completo dell'utente.
  - **private String email:** Indirizzo email utilizzato per l'autenticazione.
  - **private String passwordHash:** Password memorizzata in formato hash tramite algoritmo BCrypt.
  - **private String telefono:** Recapito telefonico dell'utente.
  - **private String indirizzo**: Indirizzo di residenza o spedizione predefinito.
  - **private String cap:** Codice di Avviamento Postale associato all'indirizzo.
  - **private boolean is_admin:** Flag che definisce se l'utente possiede i privilegi di amministratore (Gestore).
  - **private boolean isBanned:** Flag che indica se l'account dell'utente è stato sospeso.
- **Metodi:**
  - **Getter/Setter**: Metodi standard per l'accesso e la modifica degli attributi della classe.

Classe Ordine

Incapsula le informazioni generali relative a una transazione d'acquisto effettuata sulla piattaforma.

- **Attributi:**
  - **private int id:** Identificatore univoco dell'ordine.
  - **private int clienteId:** Riferimento all'identificativo dell'utente che ha effettuato l'ordine.
  - **private LocalDateTime dataOrdine:** Data e ora in cui è stata registrata la transazione.
  - **private double totale:** Valore monetario complessivo dell'ordine.
  - **private String status:** Stato corrente dell'ordine (es. "In elaborazione", "Spedito").
- **Metodi:**
  - **Getter/Setter:** Metodi standard per la gestione dell'entità.

Classe DettaglioOrdine

Rappresenta la singola voce di un ordine, memorizzando le informazioni specifiche del prodotto al momento dell'acquisto.

- Attributi:
  - **private int ordineId:** Identificativo dell'ordine correlato.
  - **private int prodottoId:** Identificativo del prodotto acquistato.
  - **private int quantita:** Numero di unità acquistate per il prodotto specifico.
  - **private double prezzoUnitario:** Prezzo di vendita del prodotto congelato al momento dell'ordine.
  - **private String indirizzo:** Indirizzo di spedizione specifico per la transazione.
  - **private String cap:** CAP di destinazione specifico per la transazione.
  - **private String nomeProdotto:** Campo ausiliario utilizzato per visualizzare il nome del prodotto nelle interfacce grafiche.
- Metodi:
  - **Getter/Setter:** Metodi standard per l'interazione con gli attributi.

Classe Recensione

Contiene il feedback testuale e la valutazione numerica forniti da un utente in merito a un prodotto.

- **Attributi:**
  - **private int id:** Identificatore univoco della recensione.
  - **private int prodottoId:** Identificativo del prodotto a cui si riferisce la recensione.
  - **private int utenteId:** Identificativo dell'utente che ha redatto la recensione.
  - **private String nomeUtente:** Campo ausiliario per visualizzare il nome del recensore nelle viste.
  - **private int voto:** Valutazione quantitativa espressa in scala da 1 a 5.
  - **private String testo:** Contenuto descrittivo del feedback.
  - **private Date dataRecensione:** Data di pubblicazione del feedback.
- **Metodi:**
  - **Getter/Setter:** Metodi standard per la gestione dell'entità.

Classe ArticoloCarrello

Rappresenta l'associazione tra un'entità prodotto e la quantità selezionata dall'utente all'interno del carrello.

- **Attributi:**
  - **private Prodotto prodotto:** Riferimento all'oggetto di classe Prodotto.
  - **private int quantita:** Numero di pezzi selezionati per l'articolo.
- **Metodi:**
  - **public double getPrezzoTotale():** Restituisce il valore totale calcolato come prodotto tra il prezzo unitario dell'articolo e la quantità selezionata.

Classe Carrello

Implementa la logica di gestione della spesa dell'utente, memorizzando l'elenco degli articoli selezionati.

- **Attributi:**
  - **private List<ArticoloCarrello> articoli:** Elenco dinamico contenente gli articoli aggiunti dall'utente.
- **Metodi:**
  - **public void aggiungiArticolo(Prodotto prodotto, int quantita):** Inserisce un nuovo articolo nel carrello o ne incrementa la quantità se già esistente.
  - **public void rimuoviArticolo(int prodottoId):** Rimuove definitivamente un articolo dal carrello in base all'identificativo del prodotto.
  - **public void aggiornaQuantita(int prodottoId, int nuovaQuantita):** Modifica il numero di unità per un dato articolo o lo rimuove se la quantità è nulla o negativa.
  - **public double getTotaleComplessivo():** Calcola la somma totale dei prezzi di tutti gli articoli presenti nel carrello.

Classe Utility: DBConnection

Responsabile della gestione della connettività con il **DBMS MySQL**, utilizza il driver **JDBC** per stabilire una comunicazione sicura e centralizzata.

- **Attributi:**
  - **URL:** Stringa di connessione al JDBC (**jdbc:mysql://localhost:3306/unisa_cardshop** nel nostro caso)
  - **USER:** Nome utente del database (es. **root**)
  - **PASSWORD:** Password per l'accesso.
- **Metodi:**
  - **public static Connection getConnection() throws SQLException:** Carica dinamicamente il driver **com.mysql.cj.jdbc.Driver** (se non è già stato caricato) e restituisce un oggetto **Connection** attivo. Questo verrà usato dalle varie implementazioni **DAO** per eseguire **query SQL**.

3.2 Package it.unisatcg.model.dao (DAO)

Interfaccia ProdottoDAO

Definisce il contratto per le operazioni di persistenza dei prodotti nel DBMS MySQL.

Metodo	Descrizione
void doSave(Prodotto prodotto)	Memorizza un nuovo prodotto nel database.
List<Prodotto> doRetrieveByCategoria(int categoriaId)	**Ricerca filtrata:** Recupera tutti i prodotti associati a una specifica categoria.
List<Prodotto> doRetrieveByCategoriaDisp(int categoriaId, String disp)	**Ricerca filtrata:** Recupera tutti i prodotti che combaciano con una data categoria e una data disponibilità
Prodotto doRetrieveByKey(int id)	Recupera un singolo prodotto tramite il suo identificativo univoco
List<Prodotto> doRetrieveAll()	Recupera l'elenco completo dei prodotti presenti nel catalogo.
List<Prodotto> doRetrieveAllDisp(String disp)	**Ricerca filtrata:** Recupera l'elenco completo dei prodotti che combaciano con la disponibilita' ricevuta come input.
void doUpdate(Prodotto prodotto)	Aggiorna le informazioni (nome, prezzo, ecc.) di un prodotto esistente.
void doUpdateQuantita(int id, int qta, Connection c)	Aggiorna la disponibilità a magazzino (usato durante la transazione di checkout).
void doDelete(int id)	Rimuove un prodotto dal catalogo tramite il suo ID.

Interfaccia CategoriaDAO

Gestisce la persistenza delle categorie necessarie per la navigazione del catalogo.

Metodo	Descrizione
void doSave(Categoria categoria)	Inserisce una nuova categoria nel sistema.
Categoria doRetrieveByKey(int id)	Recupera i dettagli di una categoria (es. “pokemon”) tramite ID.
List<Categoria>doRetrieveAll()	Recupera tutte le categorie per popolare i filtri della pagina products.jsp
void doUpdate(Categoria categoria)	Modifica il nome di una categoria esistente.
void doDelete(int id)	Elimina una categoria dal database.

Interfaccia UtenteDAO

Definisce il contratto per le operazioni di persistenza relative alla gestione dell'anagrafica utenti e delle credenziali di accesso nel sistema.

Metodo	Descrizione
void doSave(Utente utente)	Registra un nuovo profilo utente nel database.
Utente doRetrieveByEmail(String email)	Recupera i dati di un utente tramite l'indirizzo email (utilizzato per la procedura di login).
Utente doRetrieveByTelefono(String telefono)	Ricerca un utente nel sistema attraverso il numero di telefono.
Utente doRetrieveByKey(int id)	Recupera un utente specifico tramite il suo identificativo univoco.
List<Utente> doRetrieveAll()	Restituisce l'elenco completo di tutti gli utenti registrati nella piattaforma.
void doUpdate(Utente utente)	Aggiorna le informazioni anagrafiche (nome, indirizzo, ecc.) di un utente esistente.
void doDelete(int id)	Rimuove permanentemente un account utente dal database.
void updatePassword(Utente utente)	Aggiorna esclusivamente la password hash associata all'account dell'utente.

Interfaccia OrdineDAO

Gestisce la persistenza degli ordini d'acquisto, supportando sia operazioni singole che transazionali necessarie per la fase di checkout.

Metodo	Descrizione
void doSave(Ordine ordine)	Memorizza un nuovo ordine nel sistema.
void doSave(Ordine ordine, Connection con)	Registra un ordine all'interno di una transazione JDBC per garantirne l'atomicità.
Ordine doRetrieveByKey(int id)	Recupera le informazioni generali di un ordine tramite il suo ID.
List<Ordine> doRetrieveByUtente(int clienteId)	Recupera lo storico di tutti gli ordini effettuati da un determinato cliente.
List<Ordine> doRetrieveAll()	Restituisce la lista globale degli ordini presenti nel sistema per la gestione amministrativa.
void doUpdate(Ordine ordine)	Modifica i dati di un ordine esistente (es. aggiornamento del totale).
void doDelete(int id)	Elimina un record d'ordine dal database.
void updateStatus(int id, String status)	Aggiorna lo stato di avanzamento di un ordine (es. "In elaborazione", "Spedito").
List<DettaglioOrdine> doRetrieveDettagli(int ordineId)	Recupera le singole righe di dettaglio (prodotti e quantità) associate a un ordine.

Interfaccia DettaglioOrdineDAO

Consente la gestione granulare degli articoli acquistati all'interno di un ordine, includendo i dati di spedizione specifici per la transazione.

Metodo	Descrizione
void doSave(DettaglioOrdine dettaglio)	Registra una singola riga di dettaglio per un ordine.
void doSave(DettaglioOrdine dettaglio, Connection connection)	Esegue il salvataggio del dettaglio all'interno di una transazione JDBC durante il checkout.
List<DettaglioOrdine> doRetrieveByOrdine(int ordineId)	Recupera l'elenco di tutti gli articoli, quantità e prezzi associati a un ordine specifico.
List<Map<String, Object>> doRetrieveFullDetailsByOrdine(int id)	Recupera i dettagli dell'ordine includendo metadati aggiuntivi (es. nomi prodotti) tramite join SQL.

Interfaccia RecensioneDAO

Gestisce la memorizzazione e la consultazione dei feedback e delle valutazioni lasciate dagli acquirenti sui prodotti del catalogo.

Metodo	Descrizione
void doSave(Recensione recensione)	Inserisce una nuova recensione e valutazione nel database.
List<Recensione> doRetrieveByProdotto(int prodottoId)	Restituisce l'elenco completo delle recensioni associate a un prodotto specifico.

3.3 Package it.unisatcg.control (Servlet)

CategoriaProdottiServlet

Si occupa di gestire la visualizzazione principale del catalogo e il filtraggio per categoria e disponibilità.

- **Input:** La servlet può ricevere dei parametri **categoriaId** e **disp** per filtrare i risultati.
- **Logica**:
    1. Utilizza **CategoriaDAO** per recuperare l'elenco completo delle categorie necessarie per popolare il menù di filtraggio nella sezione prodotti
    2. Se i parametri **categoriaId** e **disp** sono validi interroga **ProdottoDAO.doRetrieveByCategoriaDisp** per ottenere i prodotti pertinenti
    3. Se i parametri sono assenti, invoca **prodottoDAO.doRetrieveAll** per mostrare l'intero catalogo non filtrato.
- **Metodi:**
    - **protected void doGet(HttpServletRequest request, HttpServletResponse response):** prepara gli attributi **products**, **categorie**, **categoriaSelezionata e disponibilitasel** ed effettua il forward verso **products.jsp**.

ProductDetailServlet

Gestisce la visualizzazione della scheda tecnica dettagliata di un singolo articolo.

- **Input:** Riceve obbligatoriamente il parametro id del prodotto tramite il metodo **GET**.
- **Logica:** Recupera i dati chiamando **ProdottoDAO.doRetrieveByKey.** Se il prodotto esiste, carica i dettagli (nome,desc, prezzo etc.) nell'oggetto richiesta, in caso contrario avviene un errore.
- **Metodi:**
    - **protected void doGet(HttpServletRequest request, HttpServletResponse response):** Gestisce il recupero dell'entità **Prodotto** ed esegue un forward verso **product_detail.jsp**.

ImageServlet

Servlet dedicata alla gestione e il caricamento dinamico delle immagini salvate nel DB.

- **Input:** riceve il parametro **id** del prodotto di cui si vuole visualizzare l'immagine.
- **Logica:** Si connette al DB tramte **DBConnection**, recupera la foto in formato BLOB dalla tabella prodotto e lo converte in uno stream binario in uscita. Imposta il **contentType** della risposta su **image/jpeg.**
- **Metodi:**
    - **protected void doGet(HttpServletRequest request, HttpServletResponse response):** Apre lo stream verso il DB e scrive i byte dell'immagine direttamente nell'oggetto **OutputStream** della risposta HTTP.

LogoutServlet

Gestisce la terminazione sicura della sessione utente.

- **Logica:** Invalida la sessione HTTP corrente rimuovendo tutti gli attributi associati e reindirizza l'utente alla home page del portale.
- **Metodi:**
    - **protected void doGet(...):** Esegue l'invalidazione della sessione e il redirect.

RegisterServlet

Responsabile della creazione di nuovi profili utente, garantendo l'unicità dei dati sensibili e la sicurezza delle credenziali.

- **Input:** Riceve i parametri del form di registrazione tra cui nome, email, telefono, password, indirizzo e cap.
- **Logica:**
    - Verifica l'unicità di email e telefono interrogando **UtenteDAO** (Requisiti RAD UC6).
    - Esegue l'hashing sicuro della password tramite l'algoritmo **BCrypt** con un fattore di costo adeguato.
    - Inizializza un oggetto **Utente** con i dati validati, impostando il ruolo predefinito di "Acquirente" (isAdmin = false).
    - Invocazione di **UtenteDAO.doSave** per la persistenza e reindirizzamento al login.
- **Metodi:**
    - **protected void doPost(...)**: Gestisce la sottomissione del form e la creazione dell'account.

UpdateProfileServlet

Consente agli utenti autenticati di modificare le proprie informazioni anagrafiche e di aggiornare la password.

- **Logica:** Recupera l'utente dalla sessione e ne aggiorna gli attributi con i nuovi valori ricevuti dal form. Se viene fornita una nuova password, questa viene cifrata con **BCrypt** prima dell'aggiornamento tramite **UtenteDAO.updatePassword**.
- **Metodi:**
    - **protected void doPost(...)**: Applica le modifiche al profilo nel database e aggiorna l'oggetto utente in sessione.

UserOrdersServlet

Prepara i dati relativi allo storico degli acquisti dell'utente per la visualizzazione nella sezione personale.

- **Logica:** Identifica l'utente tramite la sessione e recupera la lista di tutti i suoi ordini utilizzando **OrdineDAO.doRetrieveByUtente**. La lista viene quindi passata come attributo della richiesta alla pagina user_orders.jsp.
- **Metodi:**
    - **protected void doGet(...)**: Carica l'elenco ordini per l'utente loggato.

AdminOrderDetailServlet

Servlet dedicata alla consultazione analitica di un ordine da parte dell'amministratore, integrando dati di testata e dettagli degli articoli.

- **Logica:** Recupera le informazioni generali dell'ordine tramite **OrdineDAO.doRetrieveByKey** e la lista completa degli articoli con metadati associati (nomi prodotti) tramite **DettaglioOrdineDAO.doRetrieveFullDetailsByOrdine**.
- **Metodi:**
    - **protected void doGet(...)**: Prepara i dati completi dell'ordine per la dashboard amministrativa.

RecensioneServlet

Gestisce la sottomissione di nuovi feedback da parte degli utenti registrati su prodotti specifici.

- **Logica:** Inizializza un oggetto **Recensione** con i parametri ricevuti (prodottoId, voto, testo) e la data corrente del sistema, salvandolo nel database tramite **RecensioneDAO.doSave**. Al termine, effettua un reindirizzamento alla pagina di dettaglio del prodotto.
- **Metodi:**
    - **protected void doPost(...)**: Registra la nuova recensione verificando l'autenticazione dell'utente.

RicercaServlet

Implementa la funzionalità di ricerca testuale nel catalogo prodotti.

- **Logica:** Riceve la stringa di ricerca **Q** e interroga **ProdottoDAO.doRetrieveByNome** per ottenere gli articoli corrispondenti. In caso di query vuota, restituisce l'intero catalogo non filtrato.
- **Metodi:**
    - **protected void doGet(...):** Elabora la ricerca e inoltra i risultati alla vista products.jsp.

NewProductServlet

Consente al Gestore di inserire nuovi prodotti nel catalogo, includendo il caricamento di immagini binarie.

- **Logica:** Gestisce richieste di tipo **Multipart** per acquisire il file immagine (Part) convertendolo in un array di byte (BLOB). Utilizza **CategoriaDAO** per popolare il menu a tendina delle tipologie prodotti nella fase di inserimento.
- **Metodi:**
    - **protected void doGet(...):** Prepara il form di inserimento con l'elenco categorie.
    - **protected void doPost(...):** Salva l'oggetto prodotto nel database tramite **ProdottoDAO.doSave**.

CheckoutServlet

Gestisce la fase finale dell'acquisto, trasformando il contenuto del carrello in un ordine persistente e aggiornando le disponibilità di magazzino.

- **Input:** Recupera l'oggetto **Utente** e l'oggetto **Carrello** dalla sessione corrente.
- **Logica:**
    - Verifica l'autenticazione dell'utente e la presenza di articoli nel carrello.
    - Inizia una **transazione SQL** impostando setAutoCommit(false) per garantire l'atomicità.

    - Invocazione di **OrdineDAO.doSave** per creare la testata dell'ordine.
    - Iterazione sugli articoli del carrello per salvare i record in **DettaglioOrdineDAO** e contestuale aggiornamento delle scorte tramite **ProdottoDAO.doUpdateQuantita**.
    - Esecuzione del **commit** della transazione e svuotamento del carrello in sessione.
- **Metodi:**
    - **protected void doPost(HttpServletRequest request, HttpServletResponse response):** Coordina l'intero processo di checkout transazionale.

GestioneCarrelloServlet

Presiede alle operazioni di modifica del carrello virtuale durante la navigazione dell'utente.

- **Input:** Riceve i parametri azione (aggiungi, rimuovi, aggiorna), prodottoId e quantita.
- **Logica:**
    - Inizializza o recupera l'oggetto **Carrello** dalla sessione.
    - In base all'azione richiesta, invoca i metodi della classe **Carrello** (es. aggiungiArticolo o rimuoviArticolo).
    - Supporta risposte asincrone tramite il parametro ajax, restituendo un oggetto **JSON** con i totali aggiornati per migliorare la reattività dell'interfaccia (DG_2).
- **Metodi:**
    - **protected void doPost(HttpServletRequest request, HttpServletResponse response):** Elabora le modifiche al carrello e reindirizza alla visualizzazione o invia la risposta JSON.

AdminProductsServlet

Fornisce le funzionalità di amministrazione del catalogo riservate al Gestore.

- **Input:** Parametri action (es. "delete") e id del prodotto.
- **Logica:**
    - In fase di visualizzazione, interroga **ProdottoDAO.doRetrieveAll** per mostrare l'elenco completo degli articoli.
    - In fase di eliminazione, invoca **ProdottoDAO.doDelete** per rimuovere l'articolo dal database.
- **Metodi:**
    - **protected void doGet(...):** Carica la lista prodotti per la pagina di gestione amministrativa.
    - **protected void doPost(...):** Gestisce le azioni di modifica o cancellazione dei prodotti.

LoginServlet

Gestisce l'accesso sicuro degli utenti alla piattaforma, verificando le credenziali e lo stato dell'account.

- **Logica:** Recupera email e password dal form e interroga **UtenteDAO**. Utilizza **BCrypt.checkpw** per confrontare la password inserita con l'hash memorizzato nel database. Verifica inoltre se l'utente è stato sospeso (**isBanned**); in caso positivo, impedisce l'accesso. Se le credenziali sono corrette, inizializza la sessione e reindirizza l'amministratore alla dashboard o l'acquirente alla home page.
- **Metodi:**

- **protected void doPost(...):** Elabora la richiesta di autenticazione e gestisce il controllo del ban e dei privilegi.

AdminOrdiniServlet

Fornisce al Gestore la visualizzazione globale e il controllo degli stati di avanzamento degli ordini.

- **Logica:** In fase di visualizzazione (doGet), carica l'elenco completo delle transazioni tramite **OrdineDAO.doRetrieveAll**. In fase di aggiornamento (doPost), permette di modificare lo stato dell'ordine (es. da "Ordinato" a "Spedito") invocando il metodo updateStatus del DAO.
- **Metodi:**
  - **protected void doGet(...):** Recupera la lista ordini per la pagina admin_orders.jsp.
  - **protected void doPost(...):** Aggiorna lo stato logistico di un ordine specifico.

AdminRecensioniServlet

Modulo dedicato alla moderazione dei feedback lasciati dagli utenti sui prodotti.

- **Logica:** Recupera tutte le recensioni presenti nel sistema per la visualizzazione nella dashboard. Gestisce la rimozione di feedback ritenuti non idonei tramite il metodo **doDelete** di **RecensioneDAOImp**, garantendo il decoro del catalogo.
- **Metodi:**
  - **protected void doGet(...)**: Carica l'elenco recensioni per la gestione amministrativa.
  - **protected void doPost(...)**: Esegue la cancellazione di una recensione specifica.

EditProductServlet

Consente l'aggiornamento dinamico delle informazioni di un prodotto già esistente nel catalogo.

- **Logica:** Nella fase di caricamento (doGet), recupera i dati del prodotto tramite **ProdottoDAO.doRetrieveByKey** e l'elenco delle categorie per popolare il form di modifica. In fase di salvataggio (doPost), gestisce l'aggiornamento di prezzi, quantità e immagini multipart; se non viene caricata una nuova foto, mantiene quella preesistente recuperata dal database.
- **Metodi:**
  - **protected void doGet(...)**: Prepara i dati del prodotto per l'interfaccia di modifica edit_product.jsp.
  - **protected void doPost(...)**: Applica le modifiche all'entità prodotto nel database tramite **ProdottoDAO.doUpdate**.

3.4 Componenti di Sicurezza (Filtri)

Questa sezione descrive i componenti che implementano la logica di autorizzazione e protezione delle risorse sensibili del sistema.

AdminFilter

Filtro dedicato alla protezione delle funzionalità amministrative.

- **Logica:** Intercetta tutte le richieste verso il percorso **/admin/***. Verifica che l'utente in sessione esista e che il flag **isAdmin** sia impostato a vero. In caso negativo, esegue un reindirizzamento forzato alla pagina di login con errore di autorizzazione.

UserFilter

Garantisce che l'accesso alle aree personali sia riservato ai soli utenti autenticati.

- **Logica:** Protegge risorse quali la visualizzazione e la modifica del profilo (**/profilo.jsp, /updateProfilo**). Impedisce l'accesso ai visitatori non loggati reindirizzandoli alla procedura di autenticazione.